

Handover — how CostAdvisor actually works

Snapshot: 2026-10-01 (updated end of day). Written so somebody who has never seen this repo can be productive in a day.

`CLAUDE.md` is the authoritative, continuously-updated tracker — every scrum, what shipped, what is flagged. It is long and it is history. **This file is the mental model:** what the system is, how the pieces fit, and the handful of non-obvious rules that will cost you a day each if you learn them by breaking something.

Reading order for a new engineer: this file → `jvpdocs/local-setup.md` (get it running) → `jvpdocs/remaining-work-plan.md` (what is left, and the three places the plan's own assumptions turned out to be wrong about the data) → `CLAUDE.md` (only when you need the history of a specific scrum). `DESIGN.md` before touching any UI.

1. Read this before you touch deploy

The Railway-connected repo is not this repo, and it is stale. It is frozen at the 2026-08-22 go-live merge (`5b63c04`); `dev` is **126 commits** past that as of this snapshot. Nothing from Scrum 26 onward, **none of the 12 "Index Data Layer v2" units**, and none of the eight features in §7 is live. Do not verify anything infra-related against the live domains and conclude the code is broken — check whether that code is even deployed first. Syncing it needs someone with access to the deploy repo and the Railway/Cloudflare dashboards.

Branches

- `dev` — the only branch with the whole picture: code plus `CLAUDE.md` , `AGENTS.md` , `jvpdocs/` , `sample_idea/` , `.claude/skills/` . **Do all work here.**
- `dev-push` , `main-push` , `main` — sanitised mirrors. A strip step removes the internal docs before push, because these are exposed to the Cloudflare Workers Builds pipeline (and `main` is production). If you were handed one of these, you cannot see this file — go get `dev` .
- To update a push branch: merge `dev` , resolve every modify/delete conflict by **keeping the deletion**, then re-run the strip as a safety net: `git rm -r --ignore-unmatch .claude 49.md AGENTS.md CLAUDE.md jvpdocs sample_folder sample_idea` . `main` additionally carries its own prod infra — `frontend/wrangler.jsonc` 's Worker name is `costadvisor-web` ; do not let a merge swap it to `dev` 's `-dev` variant or a push deploys to the wrong Worker.
- Pushing to `main` / `main-push` triggers a real redeploy. Confirm with whoever owns those first.

Environments

	Production	Staging
Landing	costadvisor.org	dev.costadvisor.org
App	app.costadvisor.org	app.dev.costadvisor.org
API	api.costadvisor.org	api-dev.costadvisor.org
Branch	main	dev

Frontend: Cloudflare Workers serving the compiled SPA. Backend: FastAPI on Railway. Postgres + Redis + Celery on Railway, separate per environment. Ollama on a private Hetzner VM, reachable only over Tailscale.

2. What the product does

A supplier knows their real input costs; the buyer does not. So when the supplier says "oil went up 30%, we need +20%", the buyer has nothing to push back with. CostAdvisor closes that gap in four steps:

1. **Decompose** a product into cost components, each weighted by its share.
2. **Link** each component to a tracked commodity index, by region and quarter.
3. **Calculate** what the product *should* cost now, given how those indices actually moved.
4. **Compare** against what the supplier is charging. The difference is the negotiation signal.

Everything else in the codebase — the index layer, the catalog, the trust grading, the briefs — exists to make step 3 defensible enough to put in front of a supplier.

3. The five things to understand before changing code

3.1 There are two costing paths, and one resolver they share

This trips up everyone. The repo prices things two different ways, with two different margin conventions.

Path A — `CostModel` (a team's own product, priced against their own formula). `CostModel` → `FormulaVersion` → `FormulaComponent`. Margin is a **separate field**: the engine strips it out to get a component base, computes indexed cost, then re-applies it (`_component_base` / `_apply_margin`).

Path B — **catalog combo** (a platform `FormulaTemplate` priced in one region). `FormulaTemplate` → `FormulaTemplateComponent` + `FormulaRegionCoverage` (one row per template × region × variant, carrying the base price and period). Margin is **a line inside the recipe**, so weights legitimately sum to 99.9–110 and `evaluate_weighted_template` rebases over the recipe's own weight sum. The level is exactly 100.0 at the base period by construction.

Mixing the two conventions produces a number wrong by the margin. `coverage.margin_pct` is descriptive — applying it on top would double-count.

The bridge between them is `services/formula_resolver.get_effective_lines(db, fv, cost_model)`, which returns `(EffectiveLine[], fallback_reason)`. A `FormulaVersion` may carry `source_coverage_id + link_mode`:

- `link_mode = NULL` (unlinked) or `'pinned'` → build from the frozen `FormulaComponent` snapshot.
- `link_mode = 'tracking'` → resolve the linked catalog recipe **live**, at the cost model's current region.

All six costing entry points route through it — `_compute_indexed_cost`, `_compute_indexed_cost_detailed`, `calculate_evolution`, `calculate_brief`, `calculate_price_change`, `_compute_indexed_cost_forward`. That is why should-cost, breakdown, Evolution, Brief, Price-Change and the forward should-cost cannot disagree for the same formula version. **If you add a seventh consumer, route it through `get_effective_lines` too.**

It is also where **nested cost models** (Scrum 27) are expanded: a component of `component_type='model'` folds its child's lines in with weights multiplied. That is why nesting needed no change to `costing_engine.py` at all — every consumer inherited it. See §7 for what nesting means numerically.

3.2 The index resolution chain, in order

`data_resolver.get_single_index_value_detailed(commodity_id, region, year, quarter, team_id)` is where a number comes from. The order matters and each tier has a reason:

1. **composite** — an index computed live from other indices via an expression (cycle-guarded). A missing component yields `None`; it never fabricates a 0.
2. **fixed** — a team's `TeamIndexSource` with a constant value across all periods.
3. **team_override / provider** — `IndexOverride`, exact region then GLOBAL. **A null override is a deliberate blank**, not a miss: it returns `None` rather than falling through to the scraped value.
4. **scraped_region** — `IndexValue` at that exact region.
5. **scraped_global** — the GLOBAL sentinel row.
6. **scraped_any_region** — any region with data for that commodity and period.
7. **monthly_actual / monthly_partial_quarter** — the quarter mean of `IndexMonthlyValue` **actual** rows. Forecast rows are never used here; one reaching a historical should-cost would be fabrication. A quarter with fewer than three months is labelled partial rather than passed off as complete.
8. **scraped_temporal_carry_forward** — the last observation before this period, carried forward. This tier exists so a future reference quarter does not flatten every ratio to 1.0.
9. **monthly_carry_forward** — same, from the monthly store.

Tier 7 matters more than it looks: 76 of the 98 commodities the catalog's cost lines reference are **monthly-only**, so before that tier existed three quarters of the catalog resolved to nothing and rode flat.

A component with no value **rides flat and produces an explicit `data_gaps` entry**. It never silently becomes zero, and it is never silently omitted.

3.3 The index data model has three layers

The 2026-07 drop added a layer above the original one. Both are live.

- **TypeCode** — the label a cost line names (`ELEC-EU` , `BRENT`). Resolves to a series, or explicitly does not: `resolved` / `no_series` / `ambiguous` are three distinct states with three different remedies (run a scrape / buy a feed / decide what the code means). Never collapse them.
- **CommodityIndex** — the price series. Drop-loaded rows are identified by `commodity_key IS NOT NULL` . **Region is baked into the series key** (`ammonia-eu` vs `ammonia-in`), which is why they were not name-matched onto the older region-agnostic rows.
- **IndexCard** (display) + **IndexMonthlyValue** (the numbers). **Monthly is the source of truth; quarterly derives from it** — verified exactly derivable, 0.0000 max difference across all 1,516 quarterly rows.

Two traps: `-ppi` / `-wb` / `-mb` suffixes name a **source**, not a region — do not parse them. And `is_default_region` is deliberately **not unique**; 18 slugs ship several defaults, so the obvious constraint rejects the real data.

One number worth carrying in your head: **60 type codes resolve to `brent` , carrying ~25% of all indexed cost weight**. A breakdown that looks diversified can be one commodity reached through dozens of labels. That is what `/api/resolution` exists to show.

3.4 Tenancy is enforced at the database

Every tenant-facing table has a Postgres RLS policy (`tenant_isolation`). The active team is set per request in `app/database.py` . **Never write a query that bypasses it**. `bypass_rls_var` exists for Celery tasks, seed scripts and migrations only — and when you do set it, reset it in a `finally` .

Three tenancy shapes exist, and picking the wrong one is the most common serious bug here:

- **Strict tenant** — `team_id NOT NULL` , membership-scoped. Most things.
- **Platform-readable with team forks** — `team_id IS NULL` visible to all, team rows scoped. Used for `formula_templates` , `chemical_families` , `subfamilies` , editorial blocks, dimension terms, market signals. Under strict tenant the platform catalog would be invisible to everyone and look like a loader failure.
- **Platform-level, no RLS** — `commodity_indexes` , `producers` , calibration and projection runs. No `team_id` at all.

Two real leaks were caught this way and both had the same cause: a platform-scoped row has no team to CASCADE from, so it survives a test's tenant teardown and becomes live data in the next test's run. If you add a platform-scoped table, clean it up explicitly in tests.

3.5 Permissions: the plan ceiling runs *before* roles

`services/permissions.has_permission` evaluates in this order:

1. super-admin bypass
2. **plan ceiling** — a key absent from the team's plan is denied for everyone, whatever role they hold
3. custom team roles (`Role` → `TeamMemberRole`)

4. membership-role fallback (`owner` / `admin` / `member`), **only if no custom role is assigned**

Consequences that have bitten this codebase already:

- **A new permission key must be granted to the Dream Plan** or the team owner is locked out of the feature you just shipped. The ceiling runs first.
- **A member with any custom role skips the fallback entirely**, so role grants are not optional extras.
- The fallback used to grant every `*.view` key by splitting on the last dot, which silently opened each new sensitive category as it was added. It now checks the category against `MEMBER_READABLE_CATEGORIES` — an **opt-in** list, so a new category is closed by default.

Platform-wide permissions are a separate axis: `has_platform_permission` + `UserPlatformRole` (Chemist, FX Manager, Content Editor, Support Agent). "May you write the library everybody reads" is a different question from "may you write your own copy".

4. Repo map

Backend — `backend/app/`

FastAPI at `main.py`. 44 routers, all prefixed `/api/` except auth (`/auth/`). 74 Alembic migrations, 63 test files.

The services worth knowing by name:

File	What it owns
<code>costing_engine.py</code>	should-cost, evolution, brief, squeeze, price-change, forward should-cost. The most complex file in the repo.
<code>formula_resolver.py</code>	<code>get_effective_lines</code> , <code>flatten_components</code> , <code>evaluate_weighted_template</code> , coverage fallback. The bridge between the two costing paths.
<code>data_resolver.py</code>	the 9-tier resolution chain above, plus composites and forward values.
<code>permissions.py</code>	<code>has_permission</code> , <code>has_platform_permission</code> , the category opt-in list.
<code>resolution.py</code> / <code>proxy_derivation.py</code>	type-code resolution, concentration queries, proxy execution, the swap backlog.
<code>intelligence.py</code>	combo-grain derivation: level series, cycle position, seasonality blend, volatility percentile. Bounded query budget — 1 query regardless of window length.
<code>trigger_radar.py</code>	negotiation windows from five feeds; contract notice deadlines.
<code>trust.py</code> / <code>supplier_trust.py</code>	the combo trust grade, and supplier grading resolved through the producer master.
<code>narrative.py</code> / <code>ollama.py</code>	LLM narrative. Best-effort, Tailscale-only, <code>llm_enabled=False</code> in production.
<code>incoterm_normalizer.py</code> / <code>fx_converter.py</code> / <code>unit_converter.py</code>	the comparability pipeline. Any pricing logic must account for all three.
<code>drop/</code>	the 2026-07 data-drop reader, normalisation, authority rules, loaders.
<code>sheet_roundtrip/</code>	export → edit offline → reimport → diff → apply. Payload-agnostic; two registered payloads.
<code>price_list.py</code>	matching a supplier price-list row to a cost model (exact/fuzzy/ambiguous/unmatched) and deriving its period. Parsing is <code>quote_extraction.py</code> 's, reused unchanged.
<code>negotiation_prep.py</code>	checks a supplier's claim against the brief's own drivers. The verdict is never stored — it recomputes, because the driver's real movement changes as index data lands.
<code>ai_cost_modeler.py</code>	prompt, defensive parse, index resolution and the two promotion gates. Refuses rather than degrades.
<code>index_region_coverage.py</code>	per-region sourcing facts off <code>IndexCard</code> . A silent series field is not a disagreement.
<code>push.py</code>	web push over VAPID. Best-effort like email; a 404/410 prunes the subscription.

Data model spine. `CostModel` → `FormulaVersion` → `FormulaComponent` is the team's formula. `ChemicalFamily` → `Subfamily` → `Product/FormulaTemplate` is the catalog taxonomy. `FormulaTemplate` → `FormulaTemplateComponent` + `FormulaRegionCoverage` is the catalog recipe. `AuditLog` records every mutation and is **append-only**.

Frontend — `frontend/src/`

React 18 + React Router 6 SPA. Auth in `AuthContext.jsx` (Google OAuth + JWT in HttpOnly cookies). All HTTP goes through `api.js`. Charts are hand-written SVG components, not a chart library.

Navigation is an 8-tab journey shell: **Dashboard** → **Indexes** → **Portfolio** → **Monitor** → **Forecast** → **Negotiate** → **Intelligence** → **Team** (+ Admin for super-admins). Everything else lives in the account menu under "Go to".

Conventions, non-negotiable because the whole app follows them:

- `.ca-*` utility classes from `styles.css` plus inline styles. No CSS-in-JS, no Tailwind.
- **Every colour is a CSS variable.** Four themes switch by `data-theme`; a hardcoded hex breaks three of them. Several bugs of exactly this kind have been fixed — including `.ca-table th.right` being used 19 times and never defined, and `--danger` being referenced and never defined.
- New files go in `pages/` (full-page views), `components/` (reusable), `utils/` (pure helpers). No new top-level directories.
- `formatApiError(e)` for every error path. Raw API errors must not reach the UI.
- Read `DESIGN.md` first. The rules that catch people out: flat cards at rest (no shadow), no coloured border-strips wider than 1px, mono for every number, the 7 commodity category colours are a **data vocabulary** and never UI chrome.

5. Running it

Full instructions in `jvpdocs/local-setup.md`. Prerequisites: Postgres and Redis running locally.

```
./start.sh                                # backend :8000 + frontend :5173
cd backend && pytest                       # full suite
cd backend && alembic upgrade head          # migrations
cd frontend && npm run build                # must be clean before you commit
```

Test-environment gotchas that silently break RLS

1. **The DB role the app connects as must not be a superuser or have BYPASSRLS.** Postgres silently skips RLS for those roles, so every isolation test fails with "team B can see team A's data" while the policy is perfectly correct. If your container was created with `POSTGRES_USER=costadvisor`, that user is the cluster bootstrap superuser and **cannot demote itself**. Create a second ordinary role, grant it schema privileges, and point `DATABASE_URL` / `TEST_DATABASE_URL` at that.
2. **Two Fernet keys must be set** or the provider-credential and Google Calendar tests fail:
`PROVIDER_CREDENTIAL_ENCRYPTION_KEY`, `GOOGLE_CALENDAR_ENCRYPTION_KEY`. Generate with `python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"`.

3. **Python 3.14:** the pinned `pydantic-core` / `psycopg2-binary` / `sqlalchemy` versions have no wheels for it. Install newer versions of just those three locally — **do not change** `requirements.txt` without validating the real deploy target's Python version.
4. **The checked-in `backend/venv` is WSL-native.** On a Windows host it has `venv/bin/python` (an ELF binary) and no `venv/Scripts/python.exe`, so nothing runs from Git Bash or PowerShell. Drive it through WSL: `wsl.exe -e bash -lc 'cd /mnt/c/.../backend && ./venv/bin/python -m pytest -q'`. Postgres and Redis are inside WSL too, which is why `pg_isready` appears missing from the Windows side while the database is perfectly healthy.

Against a properly seeded database the suite is green (793 passed / 5 skipped at last full run; the skips are drop-dependent tests that skip cleanly without the data drop). Against a genuinely fresh database expect ~20 failures in `test_catalog_retarget` / `test_onboarding` / `test_seed_catalog` / `test_seed_combos` — those assume a pre-seeded DB state (the catalog workbook loaded, a super-admin seed user existing), not code bugs.

6. Conventions that will bite you

Learned the hard way, each one from a real bug in this repo.

- **Never delete on silence.** A data drop is authoritative for what it covers and silent about the rest. Rows it does not mention are reported **stale**, never removed. Every loader is idempotent by comparison, never truncate-and-reload.
- **dry_run is the caller rolling back**, not a flag threaded through the loader. One code path, so a dry run genuinely rehearses the real one.
- **Absent is not the same as empty.** A blank column in a source means "not stated" — write nothing. An early loader version treated it as "" and wiped two hand-set base prices.
- **Postgres treats every NULL as distinct in a unique constraint.** This has defeated upsert-in-place twice. If a nullable column is part of an upsert key, make it `NOT NULL DEFAULT ''` or fold it with `COALESCE` in a partial index.
- **A skipped check must never read as a clean one.** A validation check that finds nothing reports zero; it is not simply absent.
- **Never fabricate a number.** This is the product's whole value. A missing value rides flat with an explicit gap; a forecast is labelled a forecast with its method and vintage; a proxy is marked as a proxy. A fabricated "supplier's likely counter" was deleted for this reason (commit `03e0856`), and a hardcoded $\pm 1.5\%$ forecast band was removed twice.
- **Mutations are audited.** New write paths follow the `log_event` pattern. `audit_logs.team_id` is nullable — NULL means a platform-level event with no tenant, and the RLS policy keeps those invisible to tenants.
- **Alembic for every schema change.** Never modify tables by hand. Verify the downgrade too — every migration here has been up/down/up cycled.
- **A CHECK constraint can exist without being declared on the model class.** `ck_formula_components_component_type` was added by a migration, so reading `app/models/cost_model.py` says there is no constraint. If a new enum value inserts fine in your head and fails in Postgres, look for one the model never declared.

- **Correcting a migration after it has been applied does nothing.** Alembic has already recorded the revision, so the corrected body never runs — downgrade and re-upgrade. And remember the **test database is separate**: `DATABASE_URL="${APP_URL}_test" alembic downgrade -1 && DATABASE_URL="${APP_URL}_test" alembic upgrade head`. Both of these cost a debugging session each.
 - **Pydantic schemas in `app/schemas/` are the API contract.** Keep them in sync with the ORM models.
-

7. Where things stand

Every feature in every wave is built. What remains needs an account, a signature, a dashboard or a dataset — see `jvpdocs/remaining-work-plan.md` §C, or the in-app **What's left** page (account menu).

Wave 1 (sellable) — code complete. Remaining: vendor DPAs, incident-response contacts, SMTP credentials, Search Console verification, field CWV.

Wave 2 (catalog) — all scrums shipped. Remaining is data: base-price anchors for the combos, and feed mapping for the catalog commodities (FD-1).

Wave 3 (intelligence & depth) — all shipped, including all 12 units of the Index Data Layer v2 and, as of this snapshot, the last eight open items: the data-quality console (Scrum 33), real index forecasts (Scrum 21), supplier price-list import (Scrum 30), per-region sourcing (Scrum 57 follow-up), guided negotiation prep (Scrum 29), web push, nested "Lego" cost models (Scrum 27) and the AI cost modeler (Scrum 32).

7.1 Mock-data audit (2026-10-01) — nothing is fabricated on the frontend

Six features shipped briefly (2026-09-27) as hardcoded-fixture `/preview/*` mockups behind an amber `PreviewBadge`, specifically so the spec (model, endpoints, behaviour assumed) was written down before the real build: nested cost models, the AI cost modeler, supplier price-list import, negotiation prep, per-region sourcing, and web push.

Verified directly against the current code, not against commit messages:

`frontend/src/pages/preview/` no longer exists; `/preview` now routes to `WhatsLeft.jsx` (a real-gaps-only page); and every one of the six real surfaces calls its real backend endpoint —

Feature	Real surface	Real endpoint(s)
Nested cost models	<code>CostModelBuilder.jsx</code>	<code>GET /api/cost-models/{id}/nestable</code>
AI cost modeler	<code>pages/AiCostModeler.jsx</code>	<code>/api/ai-cost-modeler/drafts</code>
Price-list import	<code>pages/PriceListImport.jsx</code>	<code>/api/price-lists/*</code>
Negotiation prep	<code>pages/workspace/NegotiationPrepArea.jsx</code>	<code>/api/negotiation-prep/*</code>
Per-region sourcing	<code>pages/IndexSourcing.jsx</code>	<code>/api/indexes/region-coverage</code>
Web push	<code>components/PushNotifications.jsx</code>	<code>/api/push/*</code> + real <code>pushManager.subscribe()</code>

A repo-wide grep for `mock / fixture / dummy / PreviewBadge / fabricat*` across `frontend/src` turns up only comments confirming the opposite (design-mockup references to `sample_idea/*.html` used purely for layout, and explicit notes that a previously-fabricated feature was deleted) — no live component renders invented data as if it were real. The one piece of genuine dead code found along the way, `_unused_PermissionForm` in `pages/Admin.jsx`, is unreferenced and never rendered — harmless, worth deleting whenever that file is next touched, not a data-integrity issue.

Two honest, explicitly-labelled placeholders remain (not mocks — they say plainly they are not real yet, which is the house style, not an oversight): - The Forecast tab's composite headline-index chart has no 1:1 real-forecast substitute for its synthetic multi-index blend (§4's note in `CLAUDE.md`'s Scrum 21 entry); real per-series forecasts exist and are charted elsewhere (`ForecastArea.jsx`'s own per-commodity cards, `/index-sourcing`). - Intelligence's "Product Intelligence" tab was flagged as a persistence-dependency placeholder when written; as of this snapshot it renders a real `ContextTab` fed by the editorial-block and dimension data that landed later (Units 7–8) rather than the original placeholder text — worth a direct look before assuming the old flag still applies if you touch that file.

7.2 What remains — everything still in `CLAUDE.md`, by kind

Non-code (an account, a signature, a dashboard, or a dataset — nothing a coding session closes):

1. **Deploy sync** — the Railway-connected repo is ~126 commits behind `dev`; nothing from Scrum 26 onward is live. Needs whoever holds the Railway/Cloudflare dashboards.
2. **SMTP credentials** — no provider account chosen; invites/alerts/demo confirmations fail silently until one is.
3. **VAPID keys in production** — push works locally; needs `VAPID_PUBLIC_KEY` / `VAPID_PRIVATE_KEY` as Railway env vars.
4. **Vendor DPA list + named incident-response contacts** — the last two items before `security-posture.md` can go to a prospect.
5. **Base-price anchors** — 199 of 200 platform combos have no real price; the import tooling (per-region editor + bulk CSV) is built and waiting on the data.

6. **FD-1 feed mapping** — the remaining free-tier catalog commodities need a human to verify each live source before a scraper is wired; two guessed candidates already failed verification and were correctly left out.
7. **Google Search Console verification + field Core Web Vitals** — need real production traffic, blocked by #1.
8. **SameSite=Strict live cross-subdomain login verification** — needs a live staging round-trip, blocked by #1.
9. **Branch protection on main** — deliberately deferred; a 3-person private repo doesn't need it yet.

Small open engineering/analyst items (real, scoped, just not done):

10. **Taxonomy reconciliation** — 87 drop-vs-platform family/subfamily name mappings sit in an analyst decision queue (`taxonomy_reconciliation` sheet-roundtrip payload, `/api/sheets/taxonomy_reconciliation`) — the mechanism is built and tested, nobody has filled in the sheet yet. Nothing downstream consumes the resulting aliases until that happens.
11. **Scrum 28's structured per-component fields** — min/max/yield bounds are written inline in advanced expressions today (`clamp` / `step` / ternaries all work); a dedicated structured-field UI is only worth building alongside a redesign of the component editor, per the original ticket.
12. **SEED-2 rebuild to the 2026-07 workbook** — blocked on a matching combos data drop from whoever owns that workbook; the catalog works today with the old-shell inconsistency explicitly flagged, not silently wrong.
13. **seed_index_metadata.py residue** — marked SUPERSEDED but still runs against the pre-drop workbook, still collapsing 158 feeds onto one representative region. Only matters for that old path.
14. **proxy_derivation.derive_value is idle** — live and correct, but all 128 series carrying `proxy_logic` have a null `operation / base_index` ; nobody has configured a real spec yet. Configuration gap, not a code gap.
15. **evaluate_all_alerts Celery beat registration** — alerts only fire on-demand via the endpoint today, not nightly; the scheduling line was never added to `celeryconfig.py` 's `beat_schedule` .
16. **ProductIntelligence / NarrativeReview persistence model** — AI narratives are Redis-cached only (7-day TTL); a real review/approval workflow needs a DB-backed model first. See §7.1 for the current state of the tab this would feed.
17. **SCRUM-71 — buy-window lock/hold verdict at catalog-combo grain.** The cost-model-grain version already shipped (Scrum 22); this is a distinct, unbuilt variant for the platform catalog itself, and the ticket flags a real sequencing hazard: do not build it before a combo-grain forecast exists.
18. **CON-4 / CON-5** — the editorial-content loader (parses the raw drop JSON into `EditorialBlock` rows) and the staleness-recompute job for those rows. The schema and the read path (Unit 7) are built; the jobs that populate and refresh them are not.

Deliberately not doing, by design — do not re-open these as oversights:

- Predicting a supplier's likely counter-offer. No supplier-cost data exists to do it honestly; a fabricated version was already built and removed once (commit `03e0856`).
- Statistical cross-checking of two independent readings of one index series. We mostly hold only one reading per series; the few exceptions don't justify the machinery.
- Sheet-based BI/ERP export (SCRUM-69). Parking-Lot scope, never committed to this roadmap.

7.3 Architectural follow-ups worth knowing about before you build near them

Not bugs, not blockers — places where the current shape is a deliberate, scoped choice that the *next* piece of work in that area should know about rather than rediscover:

- **The component editor has one more axis coming.** If #11 above (structured min/max/yield fields) is ever picked up, it changes `FormulaComponentItem / FormulaComponentOut` and the Reference Index column in `CostModelBuilder.jsx` a second time in the same release cycle as nested cost models did — plan both together rather than bolting one onto the other.
- **EstimatorProposal and the AI-cost-modeler's AiCostDraft are deliberately two tables, not one.** `EstimatorProposal` is keyed `(template_id, region)` and approves into the *catalog*; `AiCostDraft` approves into a team's own `FormulaVersion`. They share the draft-then-approve *rules*, never the table — don't merge them later for "simplicity"; that was tried in planning and rejected for good reason (two approve paths behind one row).
- **The editorial-block / dimension-term layer (Units 7–8) is additive infrastructure with no owner UI yet beyond Curation's review queues.** CON-4/CON-5 (above) are the next real consumers; anything else that wants platform-authored prose or faceted tagging should read from this layer rather than inventing a second one.
- **Nested cost models and catalog template chaining are two separate recursion mechanisms with the same shape on purpose** (`formula_resolver.flatten_components` for templates, `get_effective_lines + component_type='model'` for cost models) — not yet unified, and probably shouldn't be: one is platform/team catalog data, the other is strictly tenant-owned, and RLS does not enforce the boundary between them at the FK level (see §3.4) — a future unification would need to re-derive the same-team-only write-time guard currently living in `routers/cost_models.py`.
- **The Intelligence combo-grain engine (`services/intelligence.py`) and the costing engine (`costing_engine.py`) read index values through two different code paths** (`derive()`'s own bulk `IndexValue / IndexMonthlyValue` reads vs. `data_resolver`'s per-lookup resolution chain), reconciled only by a `value_sources.matches_costing_engine` disclosure flag rather than a shared code path — a deliberate query-budget tradeoff (one engine needs $O(1)$ queries regardless of window length; the other needs the full 9-tier chain per lookup). Don't assume a bug if the two ever report different numbers for the same combo; check `value_sources.divergences` first.

Four rules the last eight features encode

Each one a newcomer would otherwise rediscover by breaking something.

1. **Nested cost models contribute composition, not price.** A `component_type='model'` line folds the child's lines into the parent with weights multiplied; the parent's `base_price` stays the anchor. Same convention as a chained `FormulaTemplate`, and what keeps weights summing to one. Cycles are refused at **save** (`assert_valid_nesting`) because a loop is unbounded recursion inside the engine. An unresolvable sub-model keeps its weight, rides flat and reports a data gap — dropping it would silently rescale everything else.
2. **A stored projection vintage goes stale against its own series.** Two of the five headline series are fitted to history that has since been overtaken, so a "projected" quarter can already be a fact. Anything consuming `/projections/latest` must drop projected points at or before the newest observation and say the vintage is behind — see `forwardOf()` in `ForecastArea.jsx`.

3. **The AI cost modeler never writes a cost model, never invents an index, and never degrades to silence.** Promotion is gated on the recipe closing at 100% and every index line binding; an unresolvable suggested feed is flagged and blocks promotion by name; an unreachable model returns 503 rather than an empty draft that reads like a considered answer.
 4. **Negotiation prep takes the supplier's position as an input.** The app holds no supplier-cost data and will not predict their counter. The should-cost has already consumed every verified index movement, so no verdict ever presents a cited driver as grounds for paying more — the verdicts differ only in *how* a claim fails. The claim is stored; the verdict is recomputed each read.
-

8. Where everything else lives

- `CLAUDE.md` — the full TODO tracker, architecture conventions, security rules, and the readiness scorecard (every non-shipped item scored ease + value with reasoning). Read before starting anything.
- `jvpdocs/remaining-work-plan.md` — what is left (nothing buildable), plus a record of the three places the plan's own assumptions turned out to be wrong about the data. Worth reading before trusting any ticket's description of a dataset.
- `jvpdocs/local-setup.md`, `development.md` — getting it running.
- `jvpdocs/wave1manual.md` — the complete non-code checklist for Wave 1.
- `jvpdocs/security-posture.md`, `eu-data-residency.md`, `backup-retention-policy.md`, `incident-response.md`, `vendor-risk.md` — security docs with explicit `[NEEDS CONFIRMATION]` / `[NEEDS ACTION]` placeholders.
- `jvpdocs/indexes-how-to-add.md` — adding a new index or scraper.
- `jvpdocs/custom_rbac.md` — the permission model in detail.
- `DESIGN.md` — the design system. Mandatory before UI work.
- `PRODUCT.md`, `technical-overview.md` — short external-facing summaries.
- `sample_idea/` — raw source material (ticket prompts, data drops, workbooks) most Wave 2/3 scrums were built against. Kept for provenance; large, not for browsing.